

New DAX Functions

functions introduced in Microsoft Power BI updates between 2023 and 2025. Designed for Power BI developers, Business Analysts, and data professionals, it covers essential syntax, use cases, and comparisons for cutting-edge windowing, statistical, and metadata functions.

Embrace the future of data modeling with these powerful additions to the DAX language, enabling more complex calculations and streamlined analytical processes.



Table of Contents and Overview

The following sections detail each new function, focusing on structure, definition, and practical application. We begin with the powerful new windowing functions, followed by advanced statistical and metadata capabilities.

1	Window Functions & Positional Lookups LOOKUP, LOOKUPWITHTOTALS, FIRST, LAST, NEXT, PREVIOUS
2	Sequencing & Matching MATCHBY, RANK, ROWNUMBER
3	Statistical and Regression Analysis LINEST, LINESTX
4	Metadata and System Information INFO.TABLES, INFO.COLUMNS, INFO.RELATIONSHIPS
5	Advanced Custom Logic User-Defined Functions (UDFs) Overview

These functions are designed to work particularly well within Visual Calculations, offering powerful new ways to analyze data sequence and context within a visual element.

The LOOKUP Function: Contextual Value Retrieval

The new LOOKUP function offers a streamlined approach to retrieving values from a table based on specified matching criteria. Unlike the older, array-style DAX LOOKUPVALUE, this function is specifically tailored for scenarios involving windowing and row context within visual calculations, allowing for cleaner, more readable DAX.



Introduced In

Late 2023 / Early 2024



Category

Windowing / Table Function



Return Value

A single scalar value.

Definition & Syntax

```
LOOKUP (
  value_columnName,
  match_columnName, match_value
  [, match_columnName, match_value ]...,
  table_expression
)
```

Parameters Explanation

- value_columnName:** The column from which the return value is retrieved.
- match_columnName, match_value:** One or more pairs defining the lookup criteria. This is the key difference from older LOOKUPVALUE.
- table_expression:** The table to search within.

Use Case

Use LOOKUP when you need to retrieve a specific, non-aggregated value from a different row within the current filter context, particularly when setting up sequential comparisons or complex conditional logic where simple filtering isn't enough.



Comparison with LOOKUPVALUE

The core difference is in parameter structure. LOOKUPVALUE requires the return column, the match column, and the match value as separate, repeated parameters. The new LOOKUP is more structured, emphasizing the explicit relationship between the match column and the desired value, often leading to less ambiguous expressions when dealing with multiple match

LOOKUPWITHTOTALS: Including Grand Totals in Lookups

LOOKUPWITHTOTALS extends the functionality of LOOKUP by explicitly enabling the retrieval of grand total or subtotal values within calculation contexts where typical row-level lookups might fail to capture the aggregated context.

Key Feature	Primary Use	Benefit
Calculates values while respecting the total context (grand total or subtotals) of the partition or visual.	Essential for ratio-to-total calculations in visual context where the denominator needs to be the overall aggregate.	Simplifies DAX for complex comparisons against aggregated results without relying on external measures or complex FILTER contexts.

Syntax and Return

```
LOOKUPWITHTOTALS (  
  value_columnName,  
  match_columnName, match_value  
  [, match_columnName, match_value ]...,  
  table_expression  
)
```

Example Use Case: Calculating Percentage of Grand Total

Imagine a scenario in a matrix visual where you want to show the contribution of a specific category's total sales against the overall company sales, even in rows showing different categories. LOOKUPWITHTOTALS enables this by letting you 'look up' the grand total sales value reliably.

Developer Note: Use this function when migrating complex ratio calculations to Visual Calculations. It ensures the integrity of your denominator across varying degrees of aggregation within the visual structure.

Positional Window Functions: FIRST, LAST, NEXT, and PREVIOUS

These functions are foundational elements of the new windowing capabilities in DAX, allowing for easy navigation and comparison across adjacent rows or the ends of a specified window (partition). They significantly simplify time-series analysis and sequential calculations.



FIRST() / LAST()

Retrieves the value from the first or last row in the window defined by the ORDERBY and PARTITIONBY expressions.



NEXT()

Retrieves the value from the row immediately following the current row in the defined window.



PREVIOUS()

Retrieves the value from the row immediately preceding the current row in the defined window.

General Syntax Pattern

```
PREVIOUS (  
  value_columnName  
  [, orderby]  
  [, blanks]  
  [, partitionby]  
  [, matchby]  
  [, relationship]  
)
```

Function	Typical Scenario
FIRST() / LAST()	Calculating the change from the start or end of a period (e.g., Year-to-Date start value).
NEXT() / PREVIOUS()	Calculating sequential differences or growth (e.g., Month-over-Month change, lag analysis).

These functions require careful definition of **ORDERBY** (to establish sequence) and **PARTITIONBY** (to define independent groups) to work correctly within the current evaluation context.

MATCHBY: Precise Row Identification in Windowing

MATCHBY is a crucial companion function used within windowing and positional functions (like FIRST, LAST, NEXT, PREVIOUS) to explicitly define which columns should be used for matching rows, thus providing robust context preservation, especially when dealing with non-unique keys or ambiguous row context.

The Need for Explicit Matching

In complex data models, simply relying on the filter context to identify adjacent rows can lead to errors. MATCHBY allows the developer to guarantee accurate sequential access by specifying the exact column or set of columns that uniquely identifies the relationship between the current row and the next/previous row, often replacing or complementing the need for PARTITIONBY.

Syntax Integration MATCHBY takes one or more columns as arguments and is nested within a function like NEXT() or PREVIOUS(). ...NEXT(..., MATCHBY(table[UniqueRowID]))	Parameter A column reference (or multiple column references) that uniquely defines the row identity within the window.	When to Use It Whenever the simple ORDERBY criteria does not uniquely define a row, or when the sequential logic relies on a specific key that might not be visible in the current visual context.
---	--	--

MATCHBY is particularly useful in environments where data might be sparse or contain duplicate values in columns used for ordering. By providing an explicit matching key, the function can correctly resolve the target row for positional lookups.

RANK and ROWNUMBER: Sequencing Data

These two functions simplify the generation of ranks and sequential numbers directly within DAX calculations, eliminating the frequent need for complex combinations of temporary tables, variables, and iterative functions to achieve simple sequencing.

RANK

Calculates the rank of the current row within a window, based on the ORDERBY column(s). It supports both skip and dense ranking methods.

Syntax: RANK(reference, orderby, [tie_type])

- **Tie Types:** SKIP (default, leaves gaps) or DENSE (no gaps).
- **Use Case:** Top N analysis, leaderboards, percentile grouping.



ROWNUMBER

Assigns a sequential row number starting from 1 to each row in the specified window, based on the ORDERBY column(s). Unlike RANK, ROWNUMBER does not handle ties by repeating the number; it assigns a unique sequential number to every row.

Syntax: ROWNUMBER(orderby, [partitionby])

- **Key Attribute:** Always returns a unique integer for each row.
- **Use Case:** Generating unique keys for temporary tables, pagination, or simulating index columns within calculations.

The addition of native RANK and ROWNUMBER functions significantly boosts performance for many common analytical scenarios, particularly those involving ordering and segmentation, by processing the sequencing logic directly within the engine.

Statistical Functions: LINEST and LINESTX

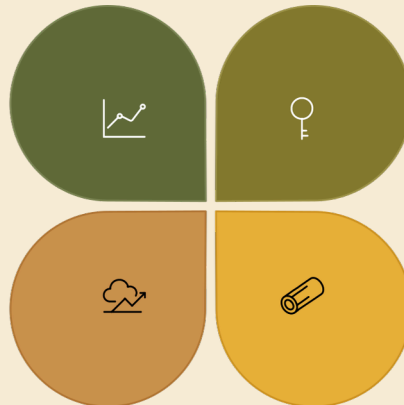
These functions bring powerful linear regression capabilities directly into the DAX modeling layer, allowing developers to perform complex statistical analysis and forecasting without exporting data or relying on external tools. They return a single row table containing the regression statistics.

LINEST

Performs linear regression against an aggregated dataset (e.g., a summarized column), returning parameters of the best-fit line.

Application

Predictive modeling, trend analysis, and establishing relationships between two variables (Y vs. X).



LINESTX

Performs linear regression row-by-row over a specified table, allowing for dynamic calculation context.

Return Table

Includes slope (m), intercept (c), R-squared value, standard error, and degrees of freedom.

Example LINESTX Syntax

```
LINESTX (  
    table,  
    y_expression,  
    x_expression  
    [, const]  
    [, stats]  
)
```

By utilizing these functions, BI professionals can operationalize advanced statistical models directly within their Power BI reports, making predictive metrics available instantly.

Metadata Functions: INFO.TABLES, INFO.COLUMNS, INFO.RELATIONSHIPS

These new informational functions allow DAX queries to retrieve structural metadata about the data model itself. While they do not manipulate data values, they are invaluable for dynamic reporting, auditing, and generating calculated table content based on model structure.



INFO.TABLES()

Returns a table containing metadata for all tables in the current model. Columns include Table Name, Hidden Status, and other properties.



INFO.COLUMNS()

Returns a table with metadata for all columns (or a specific table's columns). Includes Column Name, Data Type, and whether the column is hidden or a measure.



INFO.RELATIONSHIPS()

Returns a table detailing all relationships in the model, including 'From' Table/Column, 'To' Table/Column, direction, and activation status.

These functions are primarily used in calculated tables or for advanced model documentation. For example, a developer might create a dynamic report that lists all tables that are currently not hidden, simplifying model maintenance.



These functions are typically only accessible through tools like DAX Studio or when creating Calculated Tables and Calculated Columns in Power BI Desktop.

User-Defined Functions (UDFs)

Overview and Conclusion

The introduction of User-Defined Functions represents a paradigm shift in DAX development, allowing developers to create reusable, custom functions. This capability greatly enhances code modularity, readability, and maintenance for complex data models.

Key Characteristics of DAX UDFs

		
Modularity	Syntax	Performance
Encapsulate complex logic into a single, callable unit. Reduces redundant code across multiple measures.	Defined using the <code>DEFINE FUNCTION</code> structure, specifying input parameters and a return type.	While not guaranteed faster, UDFs often lead to cleaner query plans due to better organization of logic.

Summary of New Function Types

The functions detailed in this reference guide represent a significant evolution of the DAX language, moving it closer to a fully functional programming language suitable for advanced analytical and positional contexts.

